

Project: PulseBoard — Real-Time Collaborative Incident & Task Management Platform

Problem Statement

Modern teams often need to coordinate around incidents, technical problems, customer issues, or operational tasks. Existing systems can become fragmented across chat, spreadsheets, ticketing systems, and monitoring tools.

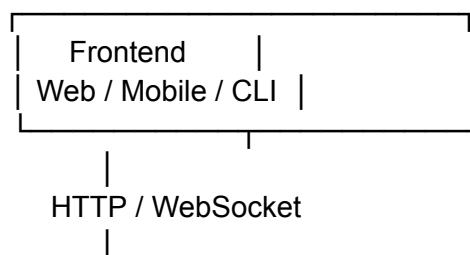
Build **PulseBoard**, a backend platform where organizations can:

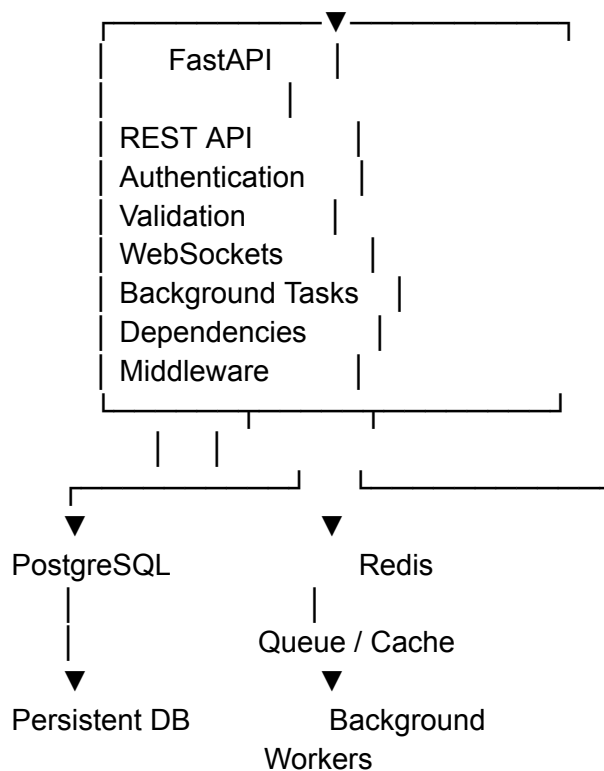
- Create projects and teams
- Create and assign tasks
- Report and manage incidents
- Track incident status in real time
- Comment on tasks/incidents
- Upload attachments
- Receive notifications
- Search and filter records
- Maintain an audit history
- Expose analytics
- Allow third-party applications to integrate through an API
- Run background jobs
- Authenticate users and enforce permissions
- Provide real-time updates through WebSockets

The system should be designed as a **production-style REST API using FastAPI**.

1. Core Architecture

Think of the system as:





You can initially run everything locally with Docker Compose.

2. User System

Users should be able to:

POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
GET /users/me
PATCH /users/me

Implement:

- Email/password authentication
- Password hashing
- JWT access tokens
- Refresh tokens
- Token expiration
- Logout/revocation
- User profile
- Account activation/deactivation

FastAPI concepts exercised

- OAuth2
 - JWT
 - Dependencies
 - Pydantic models
 - Request validation
 - HTTP status codes
 - Security dependencies
 - Headers
 - Cookies
-

3. Role-Based Access Control

Create roles:

SUPER_ADMIN
ORG_ADMIN
MANAGER
MEMBER
VIEWER

For example:

MEMBER

Can:

- View projects
- Create tasks
- Update own tasks
- Comment

MANAGER

Can:

- Assign tasks
- Create incidents
- Manage project members
- View analytics

ORG_ADMIN

Can:

- Manage organization
- Manage users
- Manage roles
- View all data

SUPER_ADMIN

Can:

- Manage everything

Implement permission dependencies such as:

```
require_authenticated_user()
require_role()
require_permission()
```

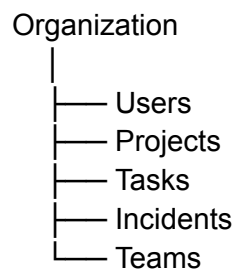
This will force you to understand one of FastAPI's most powerful concepts:

Dependency Injection.

4. Organizations

Make the application multi-tenant.

A user belongs to one or more organizations.



Endpoints:

```
POST /organizations
GET /organizations
GET /organizations/{organization_id}
PATCH /organizations/{organization_id}
DELETE /organizations/{organization_id}
```

This introduces an important backend problem:

A user must never be able to access another organization's data.

For example:

GET /organizations/123/tasks

must verify:

current_user ∈ organization_123

5. Projects

Organizations can create projects.

POST /projects

GET /projects

GET /projects/{project_id}

PATCH /projects/{project_id}

DELETE /projects/{project_id}

Project:

```
{
  "name": "Payment Gateway",
  "description": "Integration with payment provider",
  "status": "active"
}
```

Projects contain tasks and incidents.

6. Task Management

This is the primary CRUD system.

Task:

Task

- |— title
- |— description
- |— status
- |— priority
- |— project
- |— creator
- |— assignee
- |— due_date

└─ created_at
└─ updated_at

Endpoints:

POST /projects/{project_id}/tasks
GET /projects/{project_id}/tasks
GET /tasks/{task_id}
PATCH /tasks/{task_id}
DELETE /tasks/{task_id}

Statuses:

TODO
IN_PROGRESS
BLOCKED
DONE
CANCELLED

Priorities:

LOW
MEDIUM
HIGH
CRITICAL

7. Advanced Querying

This is where your API becomes more realistic.

Allow:

GET /tasks

with:

?status=IN_PROGRESS
&priority=HIGH
&assignee=123
&project=456
&created_after=...
&created_before=...
&page=2
&limit=20
&sort=-created_at

You should implement:

Pagination

```
{  
  "items": [],  
  "page": 2,  
  "limit": 20,  
  "total": 153,  
  "pages": 8  
}
```

Filtering

Sorting

Searching

Field selection

For example:

```
/tasks?fields=id,title,status,priority
```

This gives you substantial experience with FastAPI's:

- Query parameters
 - Pydantic
 - Validation
 - Dependency injection
 - Response models
-

8. Incident Management

This is what makes the project significantly more interesting.

Users can report incidents.

Example:

Payment service is returning HTTP 500 errors.

Incident:

INC-1024

Title:

Payment API failure

Severity:
SEV-1

Status:
INVESTIGATING

Affected Service:
Payments

Reported By:
Shivam

Assigned Team:
Backend

Lifecycle:

OPEN
↓
INVESTIGATING
↓
IDENTIFIED
↓
MITIGATING
↓
RESOLVED
↓
CLOSED

Endpoints:

POST /incidents
GET /incidents
GET /incidents/{id}
PATCH /incidents/{id}
POST /incidents/{id}/resolve
POST /incidents/{id}/close

9. Incident Timeline

Every incident should have an event timeline.

Example:

18:32 Incident created

18:34 Assigned to Backend Team
18:41 Status changed → INVESTIGATING
18:47 Comment added
19:02 Status changed → MITIGATING
19:18 Deployment performed
19:25 Status changed → RESOLVED

API:

GET /incidents/{id}/timeline

This introduces **event-driven thinking**.

10. Comments

Users can comment on tasks and incidents.

POST /tasks/{id}/comments

GET /tasks/{id}/comments

POST /incidents/{id}/comments

GET /incidents/{id}/comments

Support:

- Pagination
- Editing
- Deleting
- Mentions

Example:

@john please investigate the database connection pool.

11. Real-Time Updates

Now use one of FastAPI's most interesting features:

WebSockets

When someone changes an incident:

Incident status:

INVESTIGATING → RESOLVED

all connected users viewing that incident should immediately receive:

```
{  
  "event": "incident.updated",  
  "incident_id": 1024,  
  "status": "RESOLVED"  
}
```

Endpoint:

WS /ws/incidents/{incident_id}

This lets you learn:

- WebSockets
 - Connection management
 - Broadcasting
 - Authentication over WebSockets
 - Real-time event architecture
-

12. Notifications

Users should receive notifications for:

Task assigned

Task mentioned

Incident assigned

Incident severity changed

Comment added

Incident resolved

Notification API:

GET /notifications

PATCH /notifications/{id}/read

POST /notifications/read-all

13. Background Tasks

Some operations shouldn't block the API request.

For example:

POST /incidents

might trigger:

1. Save incident
2. Notify managers
3. Send email
4. Create audit record
5. Update analytics

The API should return quickly while secondary work happens asynchronously.

Start with FastAPI's:

BackgroundTasks

Then later replace it with:

Celery / RQ / Dramatiq
+
Redis

This gives you two levels of learning.

14. Email System

Implement:

POST /auth/forgot-password

POST /auth/verify-email

Emails:

Welcome email

Email verification

Password reset

Incident notification

Task assignment

Initially you can use a fake email backend:

ConsoleEmailProvider

Later:

SMTP
SendGrid
AWS SES

This teaches you how to design **service abstractions**.

15. File Uploads

Allow attachments on incidents and tasks.

POST /tasks/{id}/attachments
GET /attachments/{id}
DELETE /attachments/{id}

Use FastAPI:

UploadFile
File

Support:

PDF
PNG
JPG
CSV
TXT

Eventually store them in:

AWS S3 / MinIO

instead of the local filesystem.

16. Audit Logging

Every important action should be recorded.

Example:

User: Shivam
Action: UPDATE_INCIDENT
Entity: INC-1024
Old status: INVESTIGATING

New status: RESOLVED

IP: ...

Timestamp: ...

API:

GET /audit-logs

Only admins can access this.

This is extremely useful for learning middleware, authentication, database transactions, and security concepts.

17. Middleware

Add custom FastAPI middleware for:

Request ID

Every request receives:

X-Request-ID

Request timing

Log:

GET /tasks → 142ms

Logging

INFO request_started

INFO request_completed

ERROR request_failed

Rate limiting

For example:

100 requests/minute/user

Redis can be used for this.

18. Caching

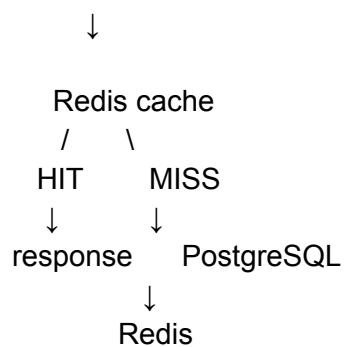
Some endpoints are expensive:

GET /analytics/dashboard
GET /projects/{id}/statistics

Cache results using Redis.

Example:

GET /projects/123/statistics



This introduces:

- Redis
 - Cache invalidation
 - TTL
 - Performance optimization
-

19. Analytics

Create:

GET /analytics/dashboard

Return:

```
{
  "total_tasks": 1523,
  "completed_tasks": 1098,
  "open_incidents": 7,
  "critical_incidents": 1,
  "average_resolution_time": 42.5,
  "tasks_by_status": {},
}
```

```
"tasks_by_priority": {},  
"incidents_by_severity": {}  
}
```

You can later build a frontend dashboard around this.

20. Health Checks

Create:

```
GET /health  
GET /health/live  
GET /health/ready
```

Example:

```
{  
  "status": "healthy",  
  "database": "healthy",  
  "redis": "healthy"  
}
```

This is useful when deploying with Docker/Kubernetes.

21. API Versioning

Don't simply create:

```
/tasks
```

Design:

```
/api/v1/tasks
```

Later:

```
/api/v2/tasks
```

This teaches you how to maintain backwards compatibility.

22. OpenAPI Documentation

One of FastAPI's major advantages is automatic documentation.

Your application should expose:

```
/docs  
/redoc  
/openapi.json
```

But don't treat this as something FastAPI does automatically and forget about it.

Use:

- Tags
- Descriptions
- Examples
- Response models
- Error responses
- Authentication schemes

so the generated documentation becomes genuinely useful.

23. Error Handling

Create consistent errors.

Instead of random responses:

```
{  
  "detail": "something went wrong"  
}
```

design:

```
{  
  "error": {  
    "code": "TASK_NOT_FOUND",  
    "message": "The requested task does not exist.",  
    "request_id": "req_8f31..."  
  }  
}
```

Create custom exception handlers for:

AuthenticationError
AuthorizationError
ResourceNotFound
ValidationError
DatabaseError
RateLimitExceeded

24. Database

Use:

PostgreSQL

ORM:

SQLAlchemy 2.x

or

SQLModel

I'd recommend **SQLAlchemy 2.x** if your goal is learning backend engineering rather than just getting the project finished.

Database:

users
organizations
organization_members
teams
projects
project_members
tasks
task_comments
incidents
incident_events
notifications
attachments
audit_logs
refresh_tokens

25. Database Migrations

Use:

Alembic

Your workflow should become:

```
alembic revision --autogenerate
alembic upgrade head
```

Don't manually modify production schemas.

26. Dependency Injection

Make heavy use of FastAPI dependencies.

For example:

```
get_db()
get_current_user()
get_current_organization()
get_project()
require_admin()
require_project_member()
```

Conceptually:

```
Request
↓
Authentication
↓
User
↓
Organization
↓
Permission
↓
Endpoint
↓
Service
↓
Repository
↓
Database
```

This is one of the most important parts of the project.

27. Pydantic

Don't use one model for everything.

Create:

TaskCreate
TaskUpdate
TaskResponse
TaskListResponse
TaskFilter

For example:

TaskCreate
 title
 description
 priority
 due_date

TaskUpdate
 title?
 description?
 priority?
 status?

TaskResponse
 id
 title
 description
 priority
 status
 created_at
 updated_at
 assignee

This will give you strong experience with Pydantic validation and serialization.

28. Testing

This is essential.

Use:

pytest
httpx
pytest-asyncio

Write:

Unit tests

test_task_service()
test_permission_check()
test_incident_state_transition()

API tests

test_create_task()
test_update_task()
test_delete_task()

Authentication tests

test_login()
test_expired_token()
test_invalid_token()

Authorization tests

test_member_cannot_delete_project()
test_user_cannot_access_other_org()

29. Async Programming

Make the application genuinely asynchronous where appropriate.

For example:

```
async def get_tasks(...):  
    ...
```

Use:

asyncpg
SQLAlchemy async engine
httpx.AsyncClient

You should understand **when async helps and when it doesn't**, rather than blindly adding **async**.

30. Docker

Your final development environment could look like:

`docker-compose.yml`

services:

api:
FastAPI

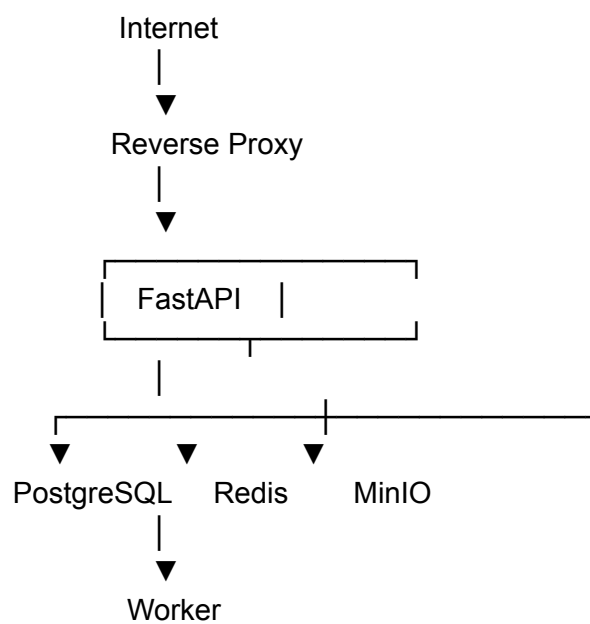
postgres:
PostgreSQL

redis:
Redis

worker:
Background worker

minio:
Object storage

Eventually:

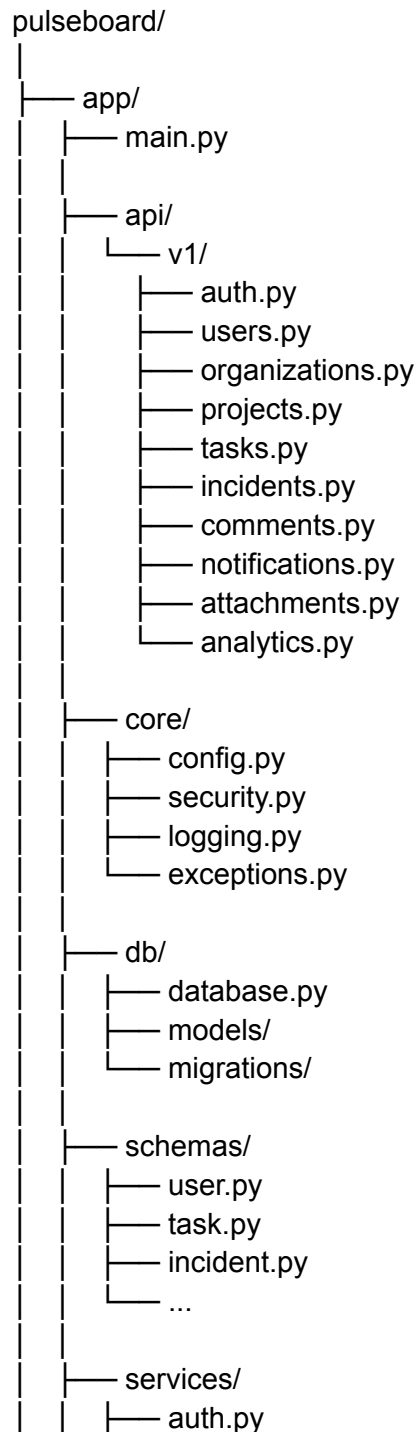


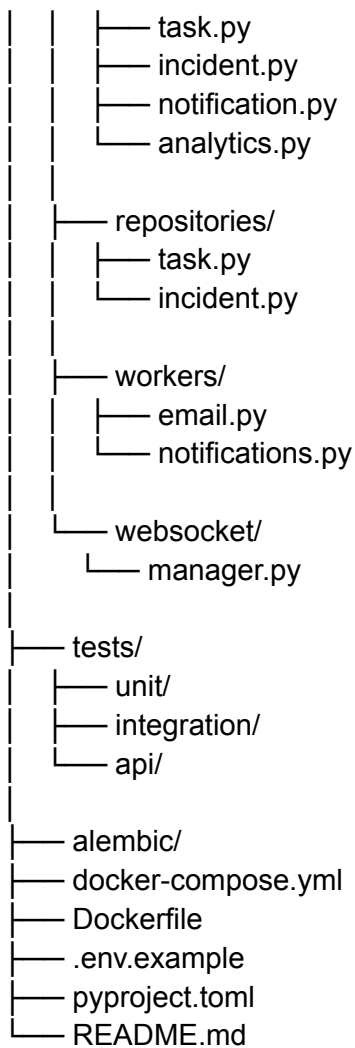
31. Suggested Repository Structure

Don't make this:

main.py
models.py
routes.py

Instead:





32. FastAPI Feature Coverage

This project is deliberately designed so that you touch a **large portion of FastAPI**.

FastAPI concept	Where you'll use it
Path parameters	<code>/tasks/{id}</code>
Query parameters	Filtering/search
Request bodies	Create/update
Pydantic	Validation
Response models	API responses
Dependencies	Auth/permissions/DB

OAuth2	Authentication
JWT	Sessions
Security	RBAC
Headers	Request ID
Cookies	Refresh tokens
File uploads	Attachments
BackgroundTasks	Notifications
WebSockets	Real-time incidents
Middleware	Logging/rate limiting
Exception handlers	Error system
Lifespan	Startup/shutdown
OpenAPI	API documentation
Async endpoints	DB/API operations
Streaming	Optional log export
CORS	Frontend
APIRouter	Modular architecture
Tags	Documentation
Status codes	REST semantics
Dependency overrides	Testing
Testing	pytest/httpx

And beyond FastAPI itself:

PostgreSQL
 SQLAlchemy
 Alembic
 Redis
 Docker
 JWT
 OAuth2
 WebSockets
 Object Storage
 Background Workers

pytest
REST API design
System architecture

33. Development Roadmap

Don't build everything simultaneously.

Phase 1 — FastAPI Fundamentals

Build:

Users
Projects
Tasks
CRUD
Pydantic
PostgreSQL
SQLAlchemy

Goal:

Understand how FastAPI communicates with a database.

Phase 2 — Authentication

Add:

Registration
Login
JWT
Refresh tokens
Password hashing

Goal:

Understand FastAPI Security + Dependencies.

Phase 3 — Authorization

Add:

Organizations

Roles
Permissions
Multi-tenancy

Goal:

Learn real backend authorization.

Phase 4 — Advanced API

Add:

Filtering
Pagination
Sorting
Searching
Validation
Custom errors

Phase 5 — Incidents

Add:

Incident management
Timeline
Comments
Audit logs
State transitions

This becomes your main domain.

Phase 6 — Async + Background Work

Add:

Redis
Background workers
Email
Notifications
Caching

Phase 7 — Real-Time

Add:

WebSockets
Live incident updates
Live notifications

Phase 8 — Files

Add:

UploadFile
MinIO/S3
Attachments

Phase 9 — Production

Add:

Docker
Health checks
Logging
Rate limiting
Environment configuration
Monitoring

Phase 10 — Testing

Target something like:

80%+ meaningful test coverage

with:

unit tests
integration tests
API tests
authentication tests
authorization tests
WebSocket tests

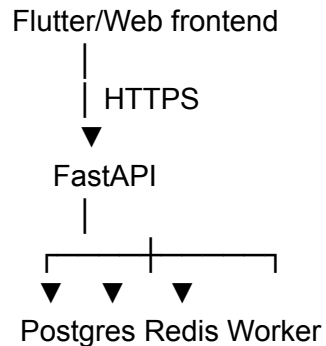
The final challenge

Once everything works, give yourself this requirement:

The frontend should not know anything about the database.

It should communicate exclusively through your FastAPI API.

For example:



Then deliberately introduce real-world constraints:

10,000 users → 100,000 tasks → 1,000 concurrent WebSocket connections → Redis caching → background workers → rate limiting → database indexes → API tests → Docker deployment.

At that point you're no longer doing a "FastAPI tutorial project." You're building a **proper backend system**, and you'll have encountered most of the FastAPI concepts that actually matter in real applications.

If your primary goal is **mastering FastAPI**, I would strongly recommend this over building another basic blog/e-commerce CRUD API.